Temporal and Contextual Matching System for Service Provider Recommendations (Availability, Location, Weather, and Time Factors) with Provider Service Demands and Peak Time Detections.

INDIVIDUAL THESIS

Service Provider Platform

Maintenance Service Provider Platform

Masakorala A H

(IT22190048)

Individual component

Temporal and Contextual Matching System for Service Provider Recommendations(Availability, Location, Weather, and Time Factors) with Provider Service Demands and Peak Time Detections.

R26-IT-072

Report submitted in partial fulfilment of the requirements for the Bachelor of Science Honours in Information Technology Specialized in Information Technology

FACULTY OF COMPUTING

Sri Lanka Institute of Information Technology

Sri Lanka

2026 OCTOBER

DECLARATION AND SUPERVISOR CERTIFICATION

I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge and belief, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic or other media, while retaining the right to use its content in future work.

The generative AI and automated assistance used in preparation of this document is disclosed in Appendix B. I remain responsible for reviewing the accuracy, originality, citations, code, data, analyses and conclusions and for completing any additional disclosure required by the module policy.

Student name -Masakorala A H
Student ID -IT22190048
Signature _____ Date _____

The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dhammika De Silva		
Co-supervisor: Samantha Rajapaksha		

Abstract

This thesis presents a service-demand estimation and provider-filtering component for a digital service platform. The forecasting branch uses 3,000 synthetic records covering fourteen categories and five districts. Date engineering and target encoding produce twenty-five model inputs. Random Forest and XGBoost are compared using a random 80/20 split and five-fold model search. Recorded holdout results favour XGBoost on R squared, RMSE and MAE: 0.882890, 3.423339 and 2.926993 respectively. Derived demand-band accuracy is 79.33 percent. Supplementary analysis shows that a category training-mean baseline reaches R squared 0.866196, demonstrating the importance of category structure in the generated task. Saved model and preprocessing artefacts generate ninety-eight next-week scenario rows; one dated CSV pair totals 1,926 predicted orders and is internally consistent. A separate notebook records a fourteen-category Firebase demand update. The filtering branch combines declared operating hours, great-circle distance and weather advice, then stores up to ten computed eligible candidates. Its captured listener run is idle rather than a completed new-request demonstration. Offline checks identify permissive malformed-time handling, reversed-window acceptance and weather-date fallback. The individual contribution is the connected modelling, scenario inference, filtering and presentation workflow, together with its evidence-based evaluation. Synthetic labels, random splitting, target encoding before inner cross-validation and fixed future covariates limit real-world forecasting claims. Future work should use observed booking histories, temporal validation, strict request checks, explicit processing states and user evaluation.

Keywords: service demand, provider filtering, XGBoost, scenario forecasting, service platform

ACKNOWLEDGEMENTS

I acknowledge the supervision and feedback provided by Dammika De Silva and Samantha Rajapaksa during this individual research component. I thank Hewapathirana NT , EMNS Roopasinghe and Tharushika W.A.M. for their valuable support, guidance, and assistance throughout the research process, including sharing useful insights and providing constructive suggestions that contributed to the improvement of this research component.

I acknowledge the use of open-source software and the research publications cited in this thesis. Generative AI assistance in document preparation and supplementary analysis is disclosed in Appendix B.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i

Abstract ii

ACKNOWLEDGEMENTS iii

TABLE OF CONTENTS iv

LIST OF TABLES ix

LIST OF FIGURES ix

LIST OF ABBREVIATIONS x

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 1

1.3 Problem Statement 2

1.4 Research Gap 3

1.5 Aim and Objectives 3

1.6 Research Questions 4

1.7 Scope Limitations and Assumptions 4

1.8 Individual Contribution 5

1.9 Thesis Structure 6

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review Method and Scope 7

2.2 Domain and Theoretical Foundations 7

2.3 Related Research 8

2.4 Existing Systems Products and Techniques 9

2.5 Gap Synthesis and Conceptual Framework 9

3 RESEARCH METHODOLOGY 11
3.1 Research Design 11
3.2 Data Participants and Study Materials 11
3.2.1 Target Construction 12
3.2.2 Realised Dataset Audit 13
3.3 Experimental and Study Procedure 14
3.3.1 Preprocessing and Validation Boundary 14
3.3.2 Scenario Forecast Procedure 15
3.4 Measures and Evaluation Metrics 15
3.5 Data Analysis 16
3.6 Ethics Privacy Security and Responsible AI 16
3.7 Reproducibility and Research Artefacts 17
4 PRODUCT AND SOLUTION DESIGN 19
4.1 Requirements 19
4.2 Component Architecture Within the Complete Solution 20
4.3 Design Alternatives and Decisions 21
4.4 Detailed Design 21
4.4.1 Forecast Features and Model Artefacts 21
4.4.2 Daily and Weekly Demand Payload 22
4.4.3 Provider Schedule Eligibility 23
4.4.4 Location and Candidate Ordering 23
4.4.5 Weather Context and Fallbacks 23
4.5 Standards and Professional Practice 24
5 IMPLEMENTATION INTEGRATION AND TESTING 25
5.1 Development Environment and Technology Stack 25
5.2 Component Implementation 25

5.2.1 Dataset Generation 25
5.2.2 Encoding and Model Comparison 26
5.2.3 Scenario Forecast and Aggregation 26
5.2.4 Filtering Listener 27
5.2.5 Web Consumers 27
5.3 Integration With the Other Components 27
5.4 Verification and Software Testing 28
5.5 Deployment and Operation 29
6 RESULTS AND VALIDATION 31
6.1 Research Results 31
6.1.1 Supplementary Baselines 32
6.1.2 Demand-Level Classification 32
6.2 Product and Technical Results 34
6.3 User or Stakeholder Validation 36
6.4 Integration Results 36
6.5 Summary of Findings 37
7 DISCUSSION 38
7.1 Interpretation of Findings 38
7.2 Comparison With Literature Baselines and Existing Solutions 38
7.3 Research Contribution 39
7.4 Product Contribution 39
7.5 Limitations and Threats to Validity 40
7.5.1 Construct and External Validity 40
7.5.2 Internal and Statistical Validity 40
7.5.3 Software and Operational Validity 41
7.5.4 Reproducibility and Attribution 41

7.6 Implications 41
8 COMMERCIALISATION AND SUSTAINABILITY 43
8.1 Value Proposition and Target Users 43
8.2 Market and Competitive Position 43
8.3 Business or Sustainability Model 43
8.4 Scalability Intellectual Property and Compliance 44
8.5 Contribution to the Sustainable Development Goals 44
9 CONCLUSION AND FUTURE DIRECTIONS 46
9.1 Conclusion 46
9.2 Achievement of Objectives 46
9.3 Individual Research and Product Contributions 47
9.4 Future Directions 47
REFERENCES 49
GLOSSARY 51
APPENDIX A EXTENDED DATA AND EVALUATION EVIDENCE 52
A.1 Category and District Counts 52
A.2 Candidate Hyperparameter Spaces 53
A.3 Supplementary Audit Procedure 54
A.4 Proposed Additional Validation 54
APPENDIX B ACADEMIC INTEGRITY AND AI USE DISCLOSURE 56
B.1 Assistance Used in Preparing This Thesis 56
B.2 Candidate Review and Attribution 56
B.3 Institutional Structure and Formatting 57
APPENDIX C TECHNICAL INTERFACES AND WORKED EXAMPLES 58
C.1 Model Feature Order 58
C.2 Worked Forecast Aggregation 59

C.3 Request Interface and Eligibility Example..... 59

C.4 Artefact Reconciliation..... 60

60

LIST OF TABLES

Table 2.1 Comparison of relevant approaches 9
Table 3.1 Dataset field groups and interpretation 12
Table 3.2 Realised demand-band distribution 13
Table 3.3 Research artefacts and evidence roles 17
Table 4.1 Component requirements and evidence status 19
Table 4.2 Principal database output contracts 22
Table 5.1 Supplementary offline schedule and distance checks 28
Table 5.2 Supplementary offline weather-rule boundaries 29
Table 6.1 Recorded holdout regression results 31
Table 6.2 Supplementary offline regression baselines 32
Table 6.3 Correctly labelled supplementary demand-band metrics 33
Table 6.4 Exported weekly scenario summary for 3 to 9 August 2026 34
Table 6.5 Recorded integration evidence and limits 36
Table 9.1 Achievement of individual objectives 46

LIST OF FIGURES

Figure 4.1 Architecture of the individual component 20
Figure 6.1 Recorded regression error comparison 31
Figure 6.2 Demand-band confusion matrix reconstructed from stored labels 33
Figure 6.3 Daily scenario predictions from the dated forecast CSV 35

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AC	Air conditioning
AI	Artificial intelligence
API	Application programming interface
CSV	Comma separated values
CV	Cross-validation
MAE	Mean absolute error
MAPE	Mean absolute percentage error
ML	Machine learning
RF	Random Forest
RMSE	Root mean squared error
RTDB	Realtime Database
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
UI	User interface
XGBoost	Extreme Gradient Boosting

1 INTRODUCTION

1.1 Overall Project Context

The project provides a digital setting in which customers can discover service providers and providers can present their services. The available application contains customer, provider and administrator views. Provider profiles include service categories, locations and working schedules, while separate views present weekly demand information and filtering history. These interfaces establish the application context for the individual component examined in this thesis [4]. The official integrated project title and the agreed allocation of shared development work require confirmation by the research group.

Two different planning questions arise within this setting. A provider may wish to understand the expected activity for a service category during the coming week. A customer may wish to identify providers who can operate during a requested date and time. The first question concerns demand estimation; the second concerns eligibility and ordering of candidates. Treating these as distinct tasks makes the component's methods and outputs easier to evaluate.

The component connects those tasks through the same service platform and database. A forecasting workflow produces category-level daily counts and weekly summaries. A filtering workflow evaluates provider schedules and geographical proximity, and adds a weather advisory for the service location. The two workflows share application infrastructure, but the available filtering code does not use the demand regressor to rank providers. That boundary is central to the interpretation of the individual contribution.

1.2 Individual Component Context

The individual component is described here as the Service Demand Prediction and Provider Filtering System. Its forecasting branch uses a synthetic dataset, feature engineering, target encoding and a comparison of Random Forest and XGBoost regressors. It saves the selected model, encoder and feature-column list so that a separate inference function can construct a next-week forecast [1], [3]. Its filtering

branch polls Firebase for new requests and returns schedule-eligible providers ordered by distance [2].

The associated React application consumes demand data through a weekly timetable and a provider category widget. An administrator history view reads filtering outputs. These files demonstrate interface implementation, although they do not establish that the candidate personally authored every application module. Individual attribution should distinguish component work from shared platform development [4].

The forecasting branch is a prototype based on generated records, not an empirically validated forecast of future customer bookings. Similarly, the filtering branch implements deterministic rules rather than a trained personalised recommendation model. The thesis evaluates the recorded experiments, technical interfaces and additional offline checks while retaining these boundaries.

1.3 Problem Statement

The component-level problem is how to provide structured service-demand estimates and operationally relevant provider candidates within one platform. A category label alone does not describe the expected activity across the coming week. A provider profile alone does not establish whether the provider's declared schedule covers a customer's requested time interval. Location and weather can also affect the interpretation of an apparently suitable candidate.

The prototype addresses this problem by constructing a daily order-count estimator and a request-processing workflow. However, generating an output is insufficient to establish that it is reliable. A demand estimate must be assessed against an appropriate target and benchmark. A filtering result must be assessed against valid schedule and location inputs. The practical problem therefore includes the consistency and evidential meaning of the output, not only the availability of a numerical score or a ranked list.

The research seeks to determine what the implemented workflow demonstrates, how its recorded model results compare, and which validation and integration limitations

must be resolved before operational use. This formulation supports concrete findings without assuming that synthetic data represent actual market demand.

1.4 Research Gap

The available component combines a demand estimation experiment with schedule, distance and weather processing. Its main research gap concerns the transition from that connected prototype to a validated service-planning system. The dataset generator defines demand through service-category base values and selected contextual adjustments. Its historical-order features are simulated, and the recorded train/test split is random. These properties limit claims about forecasting unknown future booking activity [1].

The software gap is equally specific. The filtering routine can accept unparseable or reversed request windows, uses fallback customer coordinates, and substitutes the first forecast date when the requested date is absent. Weather risk is attached as an advisory rather than an exclusion or ranking feature. No completed benchmark establishes provider-selection quality or end-to-end user acceptance [2].

The gap is therefore a need for realistic longitudinal evaluation, validated operational inputs and explicit interface semantics. The contribution of this thesis is to document the connected implementation and analyse those requirements using the actual artefacts, rather than to claim that the gap has already been fully closed.

1.5 Aim and Objectives

The aim is to develop and evaluate an individual prototype that produces category-level service-demand estimates and processes service requests into schedule-eligible, distance-ordered provider candidates with weather context.

The first objective is to create a reproducible demand dataset and define its target construction. The second is to engineer model inputs, compare Random Forest and XGBoost, and report regression and derived demand-level performance. The third is to generate a seven-day, fourteen-category scenario forecast using saved model and preprocessing artefacts. The fourth is to implement provider schedule checks, distance ordering and weather advice through a Firebase request workflow. The fifth

is to connect component outputs to the available web interfaces and identify limitations through evidence review and supplementary offline checks.

The objectives distinguish research evaluation from product implementation. A saved forecast demonstrates an inference path, while a valid future-demand claim requires later observations. A functioning schedule check demonstrates rule execution, while a robust eligibility claim requires malformed-input and boundary testing. Objective achievement is revisited in Chapter 9.

1.6 Research Questions

RQ1 asks how the recorded Random Forest and XGBoost regressors perform on the defined synthetic order-count task. RQ2 asks whether saved artefacts and inference code can produce consistent next-week daily and weekly outputs. RQ3 asks how the provider filter combines schedules, distance and weather information, and what its recorded execution demonstrates. RQ4 asks which methodological and software limitations restrict operational conclusions.

RQ1 is answered by the exported model metrics, stored predictions and additional baselines. RQ2 is answered by the dated forecast files and the demand-synchronisation notebook. RQ3 is answered through code inspection and recorded Firebase outputs; isolated helper checks examine selected edge cases. RQ4 is answered through the discussion of synthetic data, validation design, artefact differences and interface behaviour. None of these questions assumes a completed customer study.

1.7 Scope Limitations and Assumptions

The demand dataset contains 3,000 generated records, fourteen service categories, five districts and date-related, environmental and marketplace fields. Training evaluates two tree-based regressors. Next-week inference constructs fixed scenario inputs for Colombo rather than fetching actual future marketplace covariates or using the filtering weather request as a demand-model input [1], [3]. The demand forecast's spatial scope is consequently narrower than the training table's district vocabulary.

The provider filter uses declared working hours, stored coordinates and Open-Meteo weather data. It orders eligible candidates by great-circle distance and retains up to ten. It does not calculate road-route travel time, account for confirmed calendar bookings, apply a trained recommendation model, or independently authenticate provider information. Candidate-category selection is expected from supplied candidate identifiers; the fallback is the complete provider dictionary [2].

The thesis does not claim production uptime, business profitability, validated weather safety thresholds or stakeholder satisfaction. Recorded notebook outputs are historical evidence. Additional checks described in Chapters 3, 5 and 6 were performed offline during thesis preparation and do not represent a new live deployment.

1.8 Individual Contribution

The supported technical contribution is the connection between synthetic demand modelling, saved inference artefacts, a category-level forecast payload and a provider request-processing workflow. The component includes model comparison, conversion of counts into demand bands, next-week feature construction, Firebase output and web consumers. This combination provides a practical basis for later service-planning evaluation.

The individual research contribution is the analysis of the target-generation process, model selection, classification interpretation, scenario forecasting and eligibility rules. Supplementary checks strengthen the account by comparing simple baselines, correcting class-label interpretation and identifying malformed-window behaviour. These checks are disclosed separately from the original project experiments.

The candidate should confirm ownership of notebook development, scripts and frontend modules against the team's contribution records. Shared account configuration, application scaffolding and other members' components are not automatically attributable to this individual merely because they appear in the archive.

1.9 Thesis Structure

Chapter 2 reviews the relevant modelling and system concepts. Chapter 3 explains the dataset, experimental procedure, evaluation measures and evidence sources. Chapter 4 describes requirements, architecture and interface decisions. Chapter 5 documents implementation, integration and selected software checks. Chapter 6 reports numerical and technical results. Chapter 7 interprets findings and validity threats. Chapter 8 considers adoption, sustainability and proposed value. Chapter 9 concludes against the research objectives. The appendices preserve extended evidence, technical details and assistance disclosure.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review uses primary research and official documentation relevant to the implemented methods. It covers Random Forest, XGBoost, temporal evaluation, supervised categorical encoding, reproducible preprocessing, weather data and database writes [5]-[12]. Sources were selected to explain the actual component rather than to compile unrelated machine-learning techniques. The component's own notebooks and exported files provide the implementation evidence [1]-[4].

The review is a focused technical review rather than a systematic literature survey. No claim is made that every provider marketplace or forecasting method has been examined. The literature informs the interpretation of the experiment and identifies practices that should guide the next evaluation. Results reported by other authors are not transferred to this project's dataset.

2.2 Domain and Theoretical Foundations

An order-count estimator maps an input vector to a numerical target. A demand band is then derived by applying thresholds to that number. These two representations support different assessments: regression error measures numerical disagreement, while a confusion matrix measures disagreement between the derived bands. A small numerical error near a threshold can change the band; a larger error elsewhere may leave the band unchanged.

Forecasting adds a temporal condition to prediction: the information used to predict a future observation must be available at the forecast origin. Rolling-origin evaluation respects that ordering [7]. The project's random split tests generalisation across generated records but does not reproduce this deployment condition. The existence of dates in a feature table does not by itself convert a random-split experiment into temporal forecast validation.

Provider filtering has a separate theoretical basis. Eligibility is a logical condition on declared schedules and requested times; ordering is a distance comparison over

eligible candidates. Weather information contextualises the request. The implemented separation allows each rule to be inspected, but it also requires the platform to explain that schedule eligibility is not a confirmed appointment or a guarantee of service quality.

2.3 Related Research

Breiman's Random Forest combines tree predictors built with randomisation, providing the foundation for the benchmark model used here [5]. For this component, the method supplies a flexible comparison for mixed date, category and numerical inputs after encoding. Its value is established by the project's own recorded errors rather than by a general claim that forests are always superior.

Chen and Guestrin describe XGBoost as a scalable tree-boosting system with design choices supporting efficient tree learning [6]. The component applies the regressor to the same engineered inputs as Random Forest. This makes the comparison useful for selecting an estimator within the defined experiment, but does not demonstrate universal superiority or deployment readiness.

The forecasting literature's rolling-origin approach is especially relevant to future service counts [7]. It provides a design for training on earlier data and evaluating later periods. The Category Encoders documentation explains that continuous-target encoding blends category-specific target expectations with an overall expectation [8]. Because target values inform that representation, encoding must be considered part of the validation procedure.

Official scikit-learn guidance recommends fitting data-dependent preprocessing within an appropriate pipeline during cross-validation [9]. The present notebook encodes the complete training partition before its inner model search. The observed implementation therefore motivates a fold-contained encoding design in future experiments. This is a methodological recommendation, not a claim that a corrected experiment has already been performed.

2.4 Existing Systems Products and Techniques

Table 2.1 compares approaches that directly address the component tasks. Simple baselines are important because category-level base demand is built into the generator. Manual schedule review and validated deterministic filtering remain relevant alternatives to a learned provider recommender. The table compares techniques, not unverified commercial market shares.

Table 2.1 Comparison of relevant approaches

Approach	Strength for this task	Limitation or evidence needed
Global mean demand	Minimal reference for numerical prediction.	Ignores category and context.
Category mean demand	Captures persistent category differences.	Needs realistic historical observations for operational use.
Random Forest	Recorded benchmark on engineered features.	Synthetic and validation-design limits remain.
XGBoost	Selected estimator in the recorded comparison.	Small error advantage requires cautious interpretation.
Validated schedule and distance rules	Transparent eligibility and ordering.	Input validation and real scheduling evidence required.
Learned provider recommender	Could model preferences or outcomes.	Not implemented or evaluated in this component.

The component's web views add presentation and integration rather than a new prediction algorithm. A weekly timetable can make the stored counts easier to inspect; an audit history can make filtering outputs easier to review. Neither interface alone establishes the correctness or freshness of the data behind it. Source labels and timestamps should accompany presentation.

2.5 Gap Synthesis and Conceptual Framework

The review supports a framework with four linked responsibilities: construct an explicit prediction target, evaluate the estimator against relevant baselines, validate provider-request inputs, and expose outputs through consistent interfaces. A weakness in any responsibility limits the complete planning outcome. For example,

accurate synthetic prediction cannot compensate for an invalid requested time window.

The current component demonstrates a connected implementation and a recorded model comparison. Its next research step should replace simulated market histories with observations, fit encoding within temporal validation, and evaluate eligibility and user interpretation. The framework guides the design and results chapters without implying that these later studies are completed.

3 RESEARCH METHODOLOGY

3.1 Research Design

The research follows an artefact-development and evaluation design. A generated dataset supports a controlled regression experiment, and a software workflow connects predictions and provider filtering to a platform database. The analysis separates original recorded results, inspection of implementation behaviour and supplementary offline checks. This classification prevents prototype demonstrations from being treated as completed field studies.

The original experiment is documented in demand_model.ipynb and exported CSV files [1]. Provider filtering is implemented in filtering.ipynb [2]. Scenario forecasting and database synchronisation are implemented in the forecasting script and synchronisation notebook [3]. The web source establishes the consuming interfaces [4]. No training notebook or credential-dependent cloud function was executed during the supplementary analysis.

The supplementary analysis reads plain CSV files and evaluates selected pure helper functions with constructed inputs. Its purpose is to check consistency and clarify limitations. It is not a replication of the full XGBoost training process, a live forecast evaluation or a production load test.

3.2 Data Participants and Study Materials

The dataset has 3,000 rows and 23 original columns. The generator seeds Python random and NumPy with 42. Dates are drawn between 1 January 2025 and 5 July 2026. Service category, district and city are sampled from defined lists. Environmental and marketplace fields are generated through distributions or bounded random values rather than collected from customers or providers [1].

There are no recruited human participants in the recorded demand experiment. The provider and request database materials establish application integration context, but they are not presented as an independently consented research sample. Personal

names, account identifiers, precise individual locations and secret configuration contents are omitted from the thesis.

Table 3.1 groups the original fields by role. Appendix C preserves the model feature order. The target named predicted_order_count is the generated order-count label used for training; its name does not mean it was an observed prediction from a previous model. demand_level is a threshold-derived evaluation label and is excluded from the regressor input.

Table 3.1 Dataset field groups and interpretation

Group	Fields or examples	Role in the experiment
Date and calendar	date, day_of_week, month, week_of_year, weekend, holiday	Date representation and generated context.
Category and geography	service_category, district, city	Four categorical predictors including day_of_week.
Environment	rainfall_mm, temperature_c, humidity	Generated environmental inputs.
Supply and activity	available_providers, previous_day_orders, last_7_day_avg, last_30_day_avg, search_requests	Simulated marketplace context.
Outcomes and profile	completed_orders, cancelled_orders, avg_provider_rating, avg_distance_km	Additional simulated predictors.
Numerical target	predicted_order_count	Generated training label.
Demand band	demand_level	Derived evaluation label; excluded from input.

3.2.1 Target Construction

Each service category has a base demand value. Electricians have a base of 35, Plumbers 30 and Cleaners 32, while Well Services and Cushion Works have bases of 8 and 7. Rainfall above 40 mm reduces demand for selected outdoor categories and

increases it for selected other categories. Weekend adjustments apply to Vehicle Repairs, Cleaners and A/C Services. A generated holiday adds another adjustment. A final random integer between minus five and plus five is added, and the target is bounded below at one.

The target can therefore be represented as base demand plus conditional rain, weekend and holiday adjustments and random noise, followed by a lower bound. This is a developer-defined simulation. Temperature, humidity, provider availability, rating and distance are not direct terms in the target-generation rule. Historical-order fields are related to category bases but are not accumulated from a chronological booking series.

LOW is assigned at counts up to 10, MEDIUM above 10 and up to 25, and HIGH above 25. These definitions are application bands. They are not independently established market-wide capacity thresholds.

3.2.2 Realised Dataset Audit

The supplementary CSV audit found no missing values and no duplicate complete rows. The target ranges from 1 to 45, with mean 19.613 and median 18. Table 3.2 shows the realised demand-band distribution. Categories and districts occur with unequal realised frequencies despite random sampling; Appendix A reports their counts.

Table 3.2 Realised demand-band distribution

Demand band	Records	Percentage
LOW	632	21.07
MEDIUM	1410	47.00
HIGH	958	31.93
Total	3000	100.00

The audit describes the generated table only. It does not establish the prevalence of demand bands in Sri Lanka. In particular, the larger MEDIUM group follows this

generator's categories, bases and random adjustments. Sampling frequencies should not be interpreted as customer demand shares.

3.3 Experimental and Study Procedure

The notebook converts date to a datetime value and adds year, day, ISO week, numerical weekday and day-of-year fields. It overwrites month from date and removes the original date column. The original week_of_year is retained alongside the new week field. After excluding the numerical target and demand band, the model input contains 25 columns [1].

An 80/20 split with random_state 42 produces 2,400 training rows and 600 holdout rows. The split is random rather than chronological. Target encoding is fitted using the training partition and its targets, and the holdout partition is transformed without using its targets. The categorical columns are day_of_week, service_category, district and city.

RandomizedSearchCV samples 15 configurations per model family and evaluates each through five-fold cross-validation using R squared. This yields 75 cross-validation fits per family, plus a selected-estimator refit. The two selected estimators are evaluated on the holdout, and XGBoost is chosen by R squared with RMSE as the secondary ordering criterion. The precise winning parameter values are not printed in the reviewed outputs; the candidate should export them from the original environment rather than infer them from candidate lists.

3.3.1 Preprocessing and Validation Boundary

The encoder is fitted once on the full training partition before RandomizedSearchCV. Consequently, the target statistics used to encode an inner validation row incorporate targets from that inner validation partition. This creates a cross-validation leakage concern. It does not mean that holdout targets were used to fit the encoder. A later experiment should place encoding within each fold through a compatible pipeline [9].

The learning-curve plot uses three-fold evaluation on the already encoded training data. It is a training-size versus mean-squared-error diagnostic, not an epoch-by-

epoch optimisation trace. Its interpretation inherits the preprocessing concern and does not establish a causal benefit from collecting more real marketplace data.

3.3.2 Scenario Forecast Procedure

The inference function identifies the next Monday after the compile date and constructs seven dates. For a Monday reference date it advances to the following Monday, rather than the same day. Fourteen categories across seven dates produce 98 rows. Each row is encoded and aligned to the saved feature order before prediction [3].

Inputs use Colombo for district and city, rainfall 10 mm, temperature 30 degrees Celsius, humidity 75, twenty providers and category-based activity values. Holiday is zero. Selected weekend categories receive changes to previous orders and the seven-day average. These are scenario assumptions; no live historical-order aggregation or demand-side weather acquisition occurs in this function.

3.4 Measures and Evaluation Metrics

Regression evaluation uses R squared, RMSE, MAE and MAPE. RMSE and MAE express error in generated order-count units; MAPE expresses relative error and can be heavily influenced by small actual counts. R squared compares squared error against a mean-target reference. It is not a percentage of correctly classified requests. The numerical and categorical evaluations should therefore remain separate.

$$RMSE = \sqrt{\frac{\Sigma\,(y_i - p_i)^2}{n}}$$

$$R^2 = 1 - \frac{\Sigma\,(y_i - p_i)^2}{\Sigma\,(y_i - \bar{y})^2}$$

Equations 3.1 and 3.2 Regression evaluation metrics

Here n is the number of evaluated records, y_i is the true target, p_i is its prediction and $\bar{y}$ is the mean true target.

Demand bands are derived from the unrounded model outputs for the recorded classification evaluation. Accuracy is the proportion with matching bands. Precision, recall and F1 provide class-specific information. The supplementary class analysis

uses explicit labels LOW, MEDIUM and HIGH to avoid a mismatch between internal label ordering and display names [10]. Chapter 6 reports both the aggregate accuracy and the corrected per-band values.

Recorded training and prediction times describe one original execution. Search time includes the hyperparameter process and is not simply the cost of fitting a single deployed model. Prediction timing concerns the holdout batch. Hardware and execution conditions are not sufficiently documented to treat the timings as general deployment guarantees.

3.5 Data Analysis

The analysis first compares the model_comparison.csv values with notebook outputs. It then reads predictions.csv and verifies that its actual targets match the seed-42 holdout ordering from the dataset. Recomputing metrics from the two-decimal exported predictions produces small rounding differences from the original unrounded metrics. This agreement supports internal consistency without reproducing training.

Three supplementary baselines are calculated on the same split. A global training-mean baseline ignores all inputs. A category training-mean baseline estimates each service category using training targets only. A generator-category-base baseline uses the fixed bases in the simulation. These establish how much of the recorded performance can be obtained through category information alone.

The analysis also checks forecast aggregation and pure filtering helpers. It verifies daily-to-weekly consistency in the dated CSV files and probes valid, missing, malformed and reversed time windows. Weather tests use fabricated forecast dictionaries and distance tests use generic coordinates. None sends a request to the weather API or writes to Firebase. Appendix A records the procedures and evidence classification.

3.6 Ethics Privacy Security and Responsible AI

The synthetic training table contains generated records, reducing dependence on identifiable customer histories for the modelling experiment. The application

materials nevertheless include accounts, locations and credential-related configuration. The thesis omits secret contents and personal account-level examples. Public technical explanations use generic paths such as filter_requests/{request_id}, without disclosing live project addresses.

Provider availability and weather advice can affect user decisions. The interface should communicate the difference between declared schedule eligibility and a confirmed appointment. Weather thresholds in the code are local heuristics, not validated safety guidance. Missing or out-of-range forecast dates should be represented as unavailable context instead of silently using a different date.

Demand estimates should be accompanied by their compile date, target period and assumptions. Providers should not be encouraged to treat simulated counts as promised bookings. AI assistance used in preparing the thesis and the supplementary checks is disclosed in Appendix B. The candidate remains responsible for verifying technical accuracy and contribution attribution.

3.7 Reproducibility and Research Artefacts

Table 3.3 identifies the main artefacts and their evidential roles. The forecasting model, fitted target encoder and ordered columns are saved as separate files, which is more complete than saving an estimator alone. A deployment manifest should still preserve dependency versions, training configuration, source hashes and a model identifier.

Table 3.3 Research artefacts and evidence roles

Artefact	Purpose	Interpretation
demand_model.ipynb and dataset CSV	Generation, preparation and model comparison.	Original experimental method and recorded outputs.
model_comparison.csv and metrics.csv	Numerical regression results.	Authoritative recorded metric values for this account.
predictions.csv	Six hundred holdout predictions and bands.	Supports supplementary metric and class checks.
Saved model, encoder and columns	Reusable inference artefacts.	Existence does not prove a pinned runtime.

Artefact	Purpose	Interpretation
Forecast CSV files	Ninety-eight daily rows and fourteen summaries.	Dated scenario outputs with consistent aggregation.
filtering.ipynb	Schedules, distance, weather and request polling.	Implemented rules and limited recorded runtime evidence.
Synchronisation notebook and script	Weekly payload and Firebase write.	Recorded historical demand-sync demonstration.
Relevant React source	Timetable, category demand and filter history.	Interface implementation, with shared attribution to confirm.

The original notebooks, README and exported forecast files contain differences. Reproducibility requires treating each output as a dated artefact rather than merging them into a single imagined run. Chapter 6 identifies the relevant discrepancies. The supplementary analysis does not silently replace original files.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The forecasting requirements are to accept a reference date, produce seven dates for fourteen categories, preserve preprocessing consistency, and store daily counts, weekly totals, average counts and demand bands. The provider-filter requirements are to read new requests, evaluate schedules, calculate distance, add weather context, return up to ten eligible candidates and mark the request processed [2], [3].

Table 4.1 distinguishes implementation from validation. Requirements such as confirmed calendar availability, reliable malformed-input rejection and production reliability remain proposed improvements. This distinction supports a realistic design account and avoids interpreting every desirable behaviour as an existing feature.

Table 4.1 Component requirements and evidence status

Requirement	Implementation evidence	Validation boundary
Demand model comparison	Random Forest and XGBoost metrics.	Synthetic random-split evaluation.
Seven-day category forecast	Ninety-eight dated CSV rows.	Fixed scenario covariates.
Preprocessing consistency	Encoder and feature-list artefacts.	Runtime versions and manifest incomplete.
Schedule eligibility	Working-day and interval checks.	Malformed and reversed windows need correction.
Distance ordering	Haversine calculation and sorting.	Great-circle distance; no route or radius validation.
Weather advice	API acquisition and rainfall rules.	Advisory thresholds and date fallback unvalidated.
Database demand sync	Recorded write and subsequent read.	Historical demonstration, not a new live test.
Web presentation	Timetable, provider widget and history views.	No completed usability benchmark.

4.2 Component Architecture Within the Complete Solution

Figure 4.1 shows the two branches of the individual component. The training and inference branch stores a demand payload for the web views. The request branch reads provider and customer data, obtains weather context and stores filtered candidates. The connection is infrastructure and presentation; the current request branch does not consume the model's weekly demand estimates as a ranking feature.

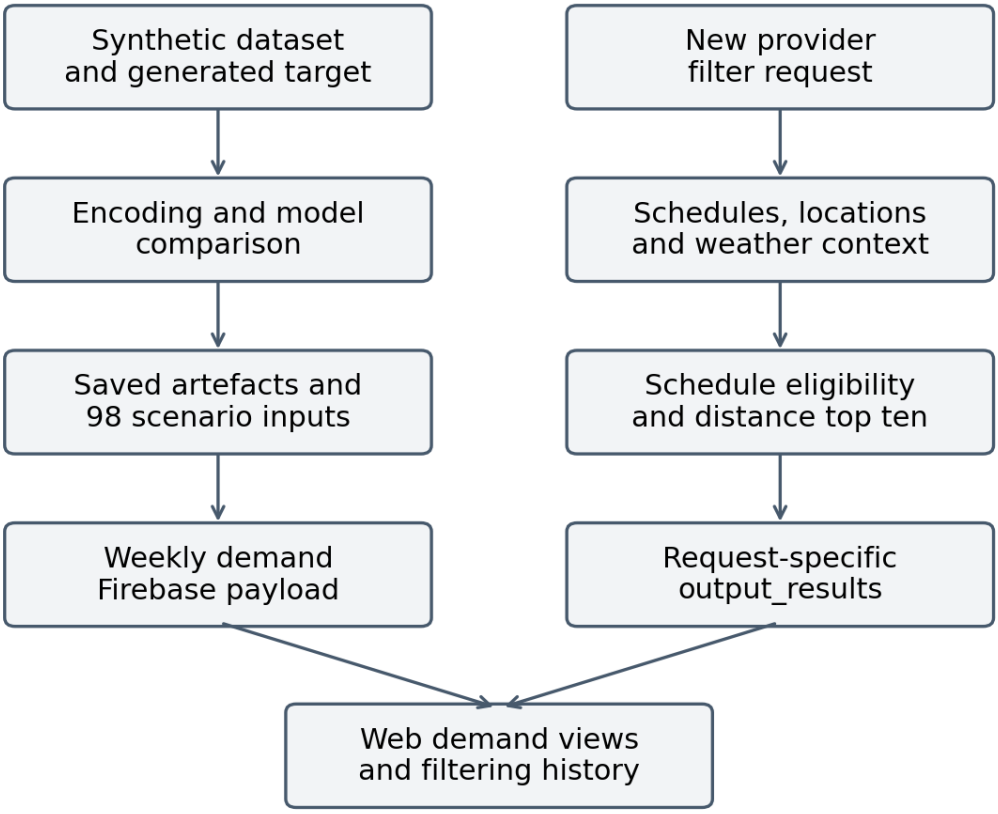


Figure 4.1 Architecture of the individual component

The wider application contains registration, document and provider-profile functionality. Those modules supply context and shared records. They should be described in the common group report as agreed shared infrastructure rather than expanded into unrelated individual research claims. The candidate's work is centred on the demand and filtering interfaces documented here.

4.3 Design Alternatives and Decisions

Tree-based regressors were selected for comparison on a tabular feature representation. The design permits nonlinear relationships and avoids manually assigning a single linear coefficient to every field. However, a category-mean baseline is especially relevant because category bases dominate target construction. The supplementary result in Chapter 6 helps assess the benefit of the selected regressor over that simpler alternative.

Target encoding maintains a compact numerical representation for the categorical fields. An alternative is one-hot encoding, which avoids category target statistics but expands the input. The implemented choice requires careful validation because its representation is supervised. The future decision should depend on leakage-controlled performance and deployment consistency, not only the size of the encoded table.

The filtering design uses explicit eligibility rules followed by distance sorting. A learned recommender would need preference or outcome labels and a separate evaluation. No such model is part of the current request routine. Retaining transparent rules is reasonable for the prototype, provided their validation and limitations are explicit.

4.4 Detailed Design

4.4.1 Forecast Features and Model Artefacts

The feature schema contains 25 columns after date engineering. During training, the four categorical fields are target-encoded. The inference function applies the saved encoder and then selects columns in the saved order. Preserving both transformation and order prevents accidental reinterpretation of input values.

The target-generation simulation and the inference assumptions are related but not identical. The generator adds random adjustments to target counts, whereas inference supplies fixed feature values and lets the learned model produce a count. Weekends modify selected historical proxies in inference rather than directly adding the

generator's random target adjustment. The resulting weekly pattern is therefore a model output under assumptions, not a fresh simulation of observed bookings.

4.4.2 Daily and Weekly Demand Payload

The count prediction is rounded and bounded below at one. Seven values are stored for each category, together with total_weekly, avg_daily and demand_level. The average is rounded to one decimal before the weekly band is assigned. The payload also includes compile_date, start_date, end_date, target_week and last_updated [3].

Table 4.2 summarises the two principal output contracts. Category keys are sanitised for Firebase by replacing certain prohibited punctuation, while service_category preserves the readable label. For example, A/C Services becomes A_C Services as an object key. This conversion should be documented so consumers do not treat the key as a different service category.

Table 4.2 Principal database output contracts

Contract	Path and fields	Consumer responsibility
Weekly demand	daily_demand/current; dates, by_category and summary	Display target period, source and assumptions.
Category entry	Seven day values, total_weekly, avg_daily and demand_level	Preserve count and band distinctions.
Filtering result	filter_requests/{request_id}/output_results	Associate output with the correct request.
Candidate list	provider_ids and evaluated_providers	Interpret schedule eligibility and distance limits.
Weather context	weather_risk, recommendation and weather_summary	Preserve date and advisory meaning.
Request status	isNewRequest set to false after output write	Define retry, claim and error semantics.

4.4.3 Provider Schedule Eligibility

The schedule checker identifies the requested weekday and reads workingHours for that day. An absent schedule or isOpen value other than true produces an unavailable result. If no request time is supplied, an open day is accepted. When all times parse, the request is accepted if its start is at or after the provider start and its end is at or before the provider end.

The comparison does not independently verify that the request end follows its start. Unparseable times are accepted when the provider day is open. These behaviours are demonstrated by supplementary checks and should be corrected through validation before the eligibility label is relied upon. Overnight schedules and existing bookings also require a different representation.

4.4.4 Location and Candidate Ordering

Haversine distance uses a spherical Earth radius of 6,371 km to approximate great-circle distance between customer and provider coordinates. This is a geographic proximity measure, not a road-network journey. Missing provider coordinates produce no distance, which is assigned an infinite sorting value. Such a provider can still enter the retained list when enough valid-distance candidates are absent.

The routine filters for the computed schedule result being true, sorts by the unrounded distance and selects the first top_k entries. Distances are rounded only for output. There is no explicit maximum-distance radius. The routine also does not apply credibility scores or validate the requested category against each fallback candidate. Category suitability therefore depends on the supplied candidate list and upstream contracts.

4.4.5 Weather Context and Fallbacks

The weather function requests current, hourly and daily data, but the risk helper uses the daily precipitation sum and temperature extrema. It searches for the service date; when the date is absent, it substitutes the first daily entry. The internal fallback marker is not included in the returned summary. This can make advice for one day appear associated with a request for another day.

Open-Meteo documents precipitation_sum as a daily precipitation aggregation and provides date arrays for the returned daily variables [11]. The API supplies weather data; the risk thresholds and indoor/outdoor advice are application rules added by this component. Keeping those roles separate prevents a data-service response from being mistaken for validation of the application's risk policy.

Outdoor and mixed-location risk is LOW at up to 5 mm, MODERATE above 5 and up to 15 mm, and HIGH above 15 mm. Indoor work receives MODERATE travel risk for rainfall above 15 mm and LOW otherwise. These thresholds are the implemented rules; the README's different thresholds are not the code's authoritative behaviour. Weather advice does not remove candidates or change their ordering.

4.5 Standards and Professional Practice

The design should preserve one explicit schema across training, inference, database storage and web consumption. Fields called demand, availability and risk need definitions that match their actual computation. A predicted count should not be described as an observed order, and a schedule-eligible candidate should not be described as a confirmed booking.

Official Firebase documentation distinguishes set, partial update and transactions [12]. The prototype uses separate output and status writes for requests. A strengthened deployment should define job claiming, idempotency and recovery for failures between those writes. The proposed changes concern the application workflow; they are not features demonstrated by the current notebook.

Professional reporting also requires provenance. Figures and tables distinguish original experiment results, dated scenario forecasts and supplementary checks. Secret configuration, personal identifiers and live account examples are omitted. The report uses numerical citations consistently and retains candidate metadata placeholders for completion.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The modelling code uses Python, pandas, NumPy, scikit-learn, Category Encoders, XGBoost, joblib, matplotlib and seaborn. Firebase Admin supports database access, and requests supports weather acquisition. The web application uses React and TypeScript with Firebase client functions and Vite tooling [1]-[4]. These are component technologies; exact installed Python dependency versions are not established by a locked environment file in the reviewed materials.

The notebooks contain warnings associated with a Python 3.8 execution environment, while the README describes a different minimum version expectation. The thesis therefore does not claim that every file was executed under a single documented runtime. A reproducible package should record the tested Python version, package versions and model compatibility together.

The web package manifest supplies intended dependency ranges, not proof of a completed build or deployment. Application execution and security-rule verification were not performed during thesis preparation. This boundary is appropriate because the available source can support interface analysis without requiring access to live accounts.

5.2 Component Implementation

5.2.1 Dataset Generation

The generator produces dates, service categories and location fields through seeded random sampling. It constructs environmental values and marketplace proxies using defined distributions and bounded values. Previous orders are sampled around category bases; recent averages are perturbations of those sampled values. They do not derive from earlier rows in a time series.

The target generator then applies conditional category, rainfall, weekend and holiday rules. Because category also influences historical proxies, several fields contain information related to the base target. This structure explains why a category-mean

baseline performs well and why feature association should not be interpreted as a discovered causal relationship in a real marketplace.

5.2.2 Encoding and Model Comparison

The encoded matrices have shapes 2,400 by 25 and 600 by 25. Model search uses the same random seed and R-squared scoring for both families. Appendix A records the candidate parameter lists. Fifteen sampled configurations are searched for each family; this is not an exhaustive search over all candidate combinations.

The selected XGBoost estimator is saved before later visual diagnostics. The notebook's learning_curve call evaluates cloned estimators at training fractions and is not a continuous training process that progressively updates the saved model. The saved encoder package contains the fitted target encoder and categorical-column names, while feature_columns.pkl preserves the input order.

The README provides different benchmark numbers and timings from the notebook and exported metrics. Table 6.1 uses the matching notebook/CSV values. A future release should regenerate documentation from the same versioned result files to prevent users from mixing experiments.

5.2.3 Scenario Forecast and Aggregation

generate_and_sync_next_week_demand constructs the next-week rows, loads the artefacts, transforms inputs and computes the payload. Its companion wrapper returns the daily dataframe, a summary dataframe and the start/end dates. The lower-level function defaults to database synchronisation enabled, while the prediction wrapper defaults to disabled [3]. This distinction matters when calling the code in an offline analysis environment.

The dated CSV pair represents 3 to 9 August 2026. It contains 98 daily rows and fourteen category summaries. A supplementary check confirmed that every summary total equals its seven daily values and that the daily file agrees with the summary. The files total 1,926 predicted orders under their fixed scenario assumptions.

5.2.4 Filtering Listener

The listener repeatedly reads customers, providers and filter_requests, checks for isNewRequest being exactly true, and processes matching requests. The configured polling interval is five seconds, followed by a sleep after processing. It is a polling implementation rather than an event-triggered database subscription.

Each request obtains customer coordinates from the account record, with hardcoded defaults when fields are absent. Candidate identifiers are taken from the request or previous result nodes; otherwise all provider keys are used. The routine obtains weather advice, evaluates schedules, sorts eligible candidates and writes the output. It then sets isNewRequest to false. There is no transaction that atomically claims the job before another worker can process it.

5.2.5 Web Consumers

WeeklyDemandTimetable normalises either by_category or the summary array, filters categories and demand bands, sorts totals and presents timetable or card views. ProviderCategoryDemandWidget looks up the provider's category through exact or approximate string matching. ProvidersFilterHistoryView consumes evaluated_providers and weather fields from output_results [4].

The Firebase client fetch functions can fall back to local backups or seed data. This supports demonstration continuity but creates a freshness concern if the interface does not clearly distinguish fallback data from live records. A rendered dashboard should not be treated as evidence of successful recent synchronisation merely because it displays a forecast.

5.3 Integration With the Other Components

The demand branch writes one shared forecast location, and the web views consume that location. The filter branch reads provider records and writes request-specific results. Shared provider metadata and workingHours are therefore important integration contracts. The demand branch's fourteen-category vocabulary should also be reconciled with the complete application's category names.

The current request routine does not use another component's credibility assessment for eligibility or ranking. A future combined workflow might use an agreed assessment field, but that would be a new design and requires evidence. The group report should describe only confirmed connections between components.

The candidate's individual integration contribution concerns these interfaces and processing functions. The complete frontend, registration system and other members' research outputs should remain attributed according to the team's agreed matrix. No complete four-component end-to-end result is inferred from the presence of shared source files.

5.4 Verification and Software Testing

Supplementary checks were performed on CSVs and selected pure helpers. Table 5.1 records schedule and distance behaviour using constructed examples. The tests clarify current behaviour; they do not modify the supplied code or establish comprehensive coverage. The provider schedule in these cases is Monday, 08:00 to 18:00.

Table 5.1 Supplementary offline schedule and distance checks

Case	Observed result	Interpretation
Monday request 10:00 to 12:00	Available	Fits declared operating window.
Monday request 07:00 to 12:00	Unavailable	Begins before provider opening.
Monday request 17:00 to 10:00	Available	Reversed request window is not rejected.
Unparseable start and end	Available	Open-day fallback is permissive.
Missing request time	Available	Open day is accepted without interval validation.
Tuesday without a schedule	Unavailable	Missing working day is rejected.
Identical generic coordinates	Distance 0 km	Basic distance identity check.

Case	Observed result	Interpretation
Missing provider coordinate	No distance	Caller sorts this value last rather than excluding it.

Table 5.2 records selected weather boundaries. A request date absent from the supplied daily data was also tested; the helper returned advice for the first available date. These findings support explicit date-coverage and input-validation requirements.

Table 5.2 Supplementary offline weather-rule boundaries

Daily precipitation	Outdoor or mixed location	Indoor location
5.0 mm	LOW_RISK	LOW_RISK
5.1 mm	MODERATE_RISK	LOW_RISK
15.0 mm	MODERATE_RISK	LOW_RISK
15.1 mm	HIGH_RISK	MODERATE_RISK
67.8 mm	HIGH_RISK	MODERATE_RISK

The current filter is called strict because only computed is_available values equal to true are retained. That statement describes a final boolean check, not strict validation of every input used to compute the boolean. The distinction explains why malformed-window tests can still yield an eligible candidate.

5.5 Deployment and Operation

The recorded demand synchronisation notebook prints a successful write and subsequently reads a current payload with fourteen categories. This supports one historical demonstration of the demand path. It does not establish a deployment schedule, monitoring process or availability service level [3].

The filter notebook reports 64 request nodes, zero new requests and an idle listener that was stopped by the user. It therefore demonstrates connectivity and polling in that execution, but not a newly completed filtering request. Separate persistent result

records should be evaluated with their own provenance before making broader operational claims [2].

A deployable service needs pinned dependencies, securely supplied credentials, logging without personal data, health checks, job-state handling and a recovery procedure. Forecasts should be versioned by target week rather than only replacing current. The component should also expose whether displayed data are live, cached or seeded. These are deployment priorities rather than completed features.

6 RESULTS AND VALIDATION

6.1 Research Results

Table 6.1 reports the matching original notebook and CSV metrics. XGBoost has slightly higher R squared and lower RMSE and MAE. Random Forest has the lower MAPE. The recorded difference supports the selected-estimator choice for this experiment, but no repeated-run uncertainty or significance analysis establishes a general ranking.

Table 6.1 Recorded holdout regression results

Model	R squared	RMSE	MAE	MAPE percent
Random Forest	0.880715	3.454984	2.967926	25.201550
XGBoost	0.882890	3.423339	2.926993	26.049276

The RMSE difference is approximately 0.032 generated orders, or 0.92 percent relative to Random Forest. XGBoost's MAPE is approximately 0.848 percentage points higher. These observations favour reporting the full metric set rather than describing one model as better in every respect. Figure 6.1 presents the numerical comparison without relabelling R squared as classification accuracy.

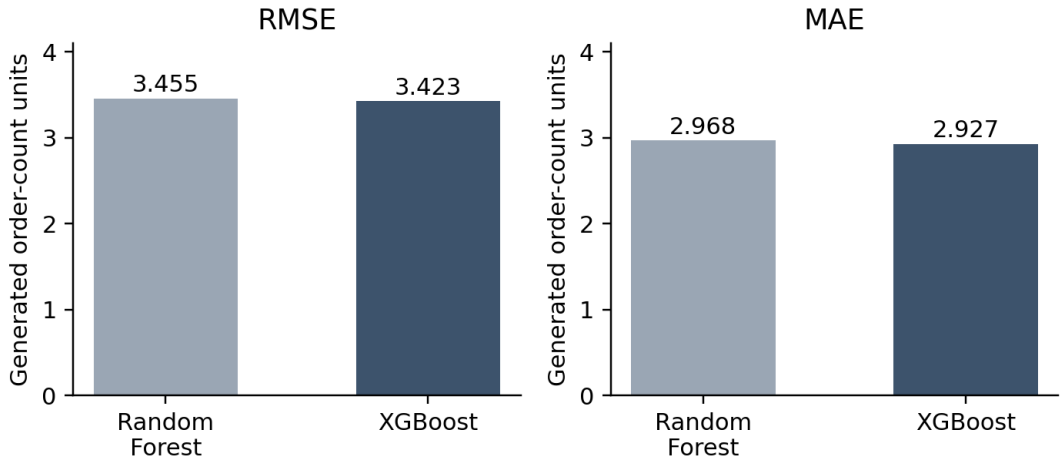


Figure 6.1 Recorded regression error comparison

Search times were 203.876 seconds for Random Forest and 39.287 seconds for XGBoost. Holdout prediction times were 0.139831 and 0.064976 seconds respectively. These are recorded batch measurements under an incompletely specified environment; they are not guaranteed response times for the integrated service.

6.1.1 Supplementary Baselines

Table 6.2 uses the same seed-42 split and training-only statistics where applicable. The global mean performs poorly compared with either model. The category training mean is much stronger, reaching R squared 0.866196 and RMSE 3.659218. XGBoost's recorded RMSE is about 6.45 percent lower than that baseline, indicating a modest additional benefit over category-level estimation on this generated task.

Table 6.2 Supplementary offline regression baselines

Baseline	R squared	RMSE	MAE
Global training mean	-0.007234	10.039648	8.654042
Category training mean	0.866196	3.659218	3.069795
Generator category base	0.859732	3.746554	3.113333

Recalculation from rounded exported XGBoost predictions yielded R squared 0.882888, RMSE 3.423371 and MAE 2.926933. These values agree closely with the original unrounded metrics. The small differences are consistent with two-decimal prediction export. This is a supplementary consistency check, not a newly trained model result.

6.1.2 Demand-Level Classification

The original aggregate band accuracy is 79.33 percent: 476 of 600 holdout records match their actual band. Figure 6.2 reconstructs the confusion matrix from the stored class strings using the explicit order LOW, MEDIUM and HIGH. Table 6.3 provides the corresponding per-band metrics.

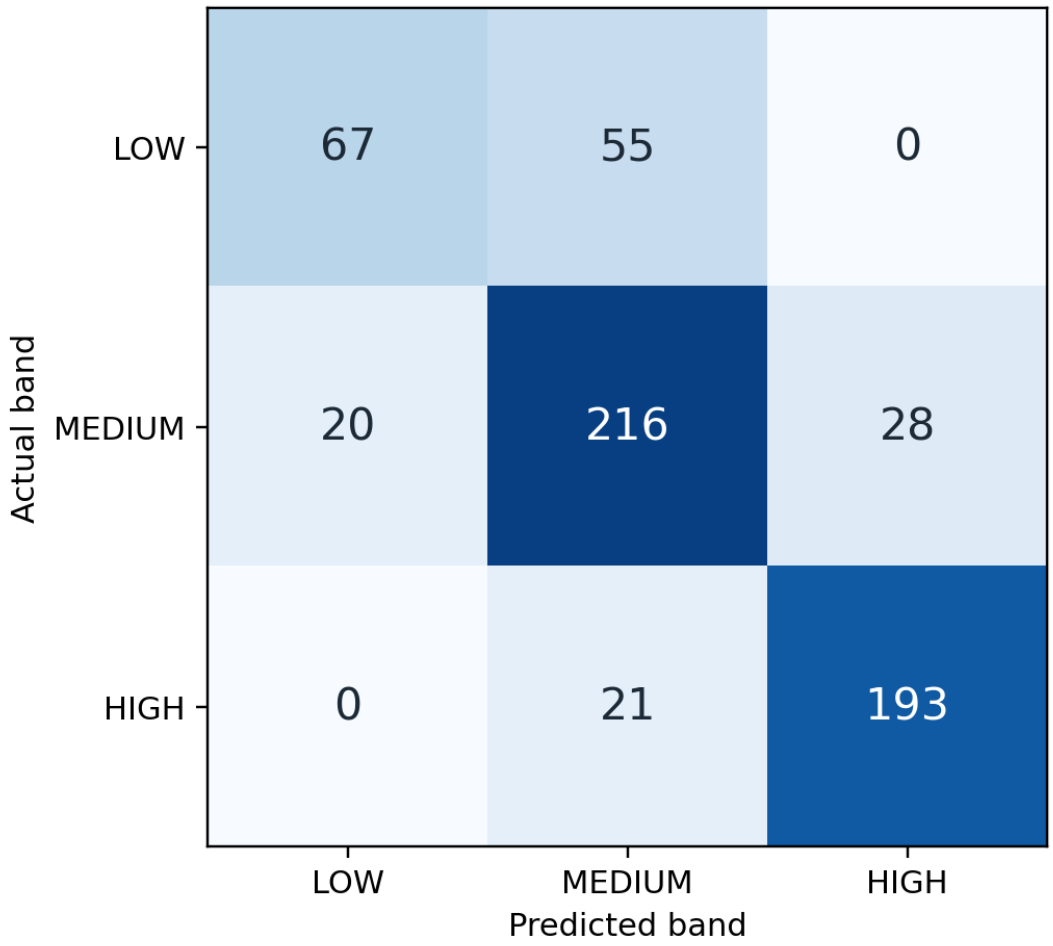


Figure 6.2 Demand-band confusion matrix reconstructed from stored labels

Table 6.3 Correctly labelled supplementary demand-band metrics

Band	Precision	Recall	F1	Support
LOW	0.770	0.549	0.641	122
MEDIUM	0.740	0.818	0.777	264
HIGH	0.873	0.902	0.887	214

The notebook passes target_names in LOW, MEDIUM, HIGH order to classification_report without explicitly passing labels. The printed row names are therefore misaligned with the default alphabetical label order. The confusion-matrix call does specify labels and is not affected by that naming issue. The supplementary

table corrects the interpretation without changing the original aggregate accuracy [1], [10].

LOW recall is the weakest of the three bands: 55 of 122 LOW cases are labelled MEDIUM. MEDIUM has errors in both adjacent directions, while HIGH has 21 cases labelled MEDIUM. There are no LOW-to-HIGH or HIGH-to-LOW errors in this stored result. These details are more informative than the aggregate percentage alone.

6.2 Product and Technical Results

Table 6.4 reports the exported scenario summary for 3 to 9 August 2026. The seven-day CSV and summary agree internally. Counts are model-generated scenario outputs, not observed orders or evaluated future outcomes. Electricians have the largest total at 240; Cushion Works and Well Services each have 56.

Table 6.4 Exported weekly scenario summary for 3 to 9 August 2026

Service category	Weekly total	Daily average	Band
Electricians	240	34.3	HIGH
Cleaners	235	33.6	HIGH
Plumbers	212	30.3	HIGH
Technicians	193	27.6	HIGH
Vehicle Repairs	181	25.9	HIGH
A/C Services	167	23.9	MEDIUM
Painters	121	17.3	MEDIUM
Carpenters	112	16.0	MEDIUM
Tile Workers	94	13.4	MEDIUM
CCTV Installation	91	13.0	MEDIUM
Masons	91	13.0	MEDIUM
Ceiling Works	77	11.0	MEDIUM

Service category	Weekly total	Daily average	Band
Cushion Works	56	8.0	LOW
Well Services	56	8.0	LOW

Figure 6.3 shows the daily values from that same CSV pair. Some categories have nearly constant predictions because most scenario inputs are fixed. Weekend variation in selected categories reflects the constructed inputs and model response. It should not be described as a discovered pattern of actual customer behaviour.

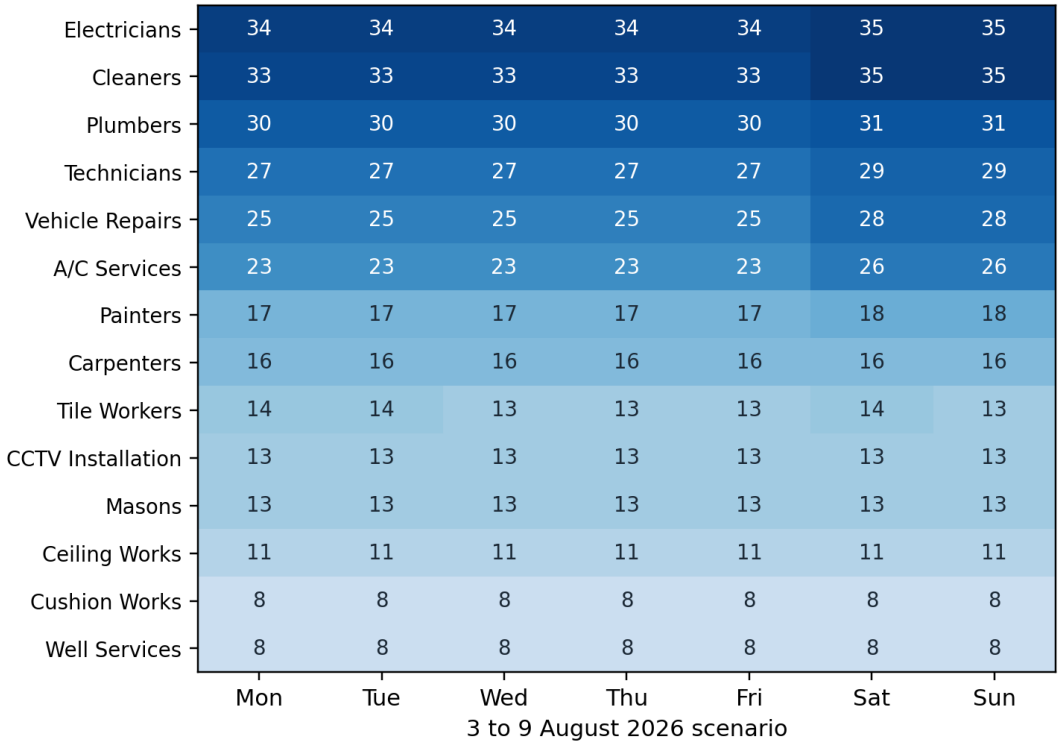


Figure 6.3 Daily scenario predictions from the dated forecast CSV

The demand-model notebook displays a Tile Workers total of 98 for the same target week, whereas the exported CSV pair contains 94. Other captured artefacts may belong to different executions. The thesis uses the internally consistent CSV pair for Table 6.4 and Figure 6.3, and retains the discrepancy as a reproducibility finding rather than combining values from separate runs.

6.3 User or Stakeholder Validation

No completed customer survey, provider interview, usability study or stakeholder acceptance dataset is recorded for this component. The implemented web controls establish the availability of presentation functions, but do not establish that users understand demand bands, trust eligibility labels appropriately or make better decisions.

A later study should ask providers to interpret a dated forecast and identify its assumptions. Customers should distinguish schedule eligibility from confirmed availability. Administrators should identify a failed weather lookup or an invalid request rather than assuming every displayed result is current. Participant selection, tasks and consent should be documented before reporting user outcomes. These are proposed validation activities.

6.4 Integration Results

The demand synchronisation notebook uses a compile date of 16 August 2026 and targets 17 to 23 August 2026. It records a successful write to daily_demand/current and a subsequent read showing fourteen category entries. The Electricians entry has a weekly total of 241, average 34.4 and HIGH band [3]. This is a different dated forecast from Table 6.4.

The filtering notebook connects to Firebase, reports 64 request nodes, finds zero new requests and displays an idle polling pass before termination. No processed-request success is recorded in that listener output. Table 6.5 distinguishes the supported integration claims from the remaining evidence needs.

Table 6.5 Recorded integration evidence and limits

Integration path	Recorded evidence	Limit
Demand synchronisation	Successful write message and subsequent fourteen-category read.	One historical execution; not a new live test.
Electricians demand entry	Total 241 and HIGH for 17 to 23 August.	Scenario output; no observed-order comparison.

Integration path	Recorded evidence	Limit
Filter connectivity	Sixty-four nodes read, zero new requests.	No new request processed in this output.
Filter listener	Idle polling pass and user termination.	No throughput or uptime measurement.
Web consumption	Demand and filtering fetch functions and views.	Source implementation; no completed runtime or usability test.

The provider filter's weather advisory and ranking path are established by source inspection and helper checks, not by an invented end-to-end request result. Future integration evidence should include an anonymised constructed request, expected candidate list, observed database payload and display confirmation from the same run.

6.5 Summary of Findings

The research results support a functioning synthetic regression workflow and a selected XGBoost estimator. A strong category-mean baseline shows that much of the performance derives from the generator's category structure. The demand-band analysis supports 79.33 percent aggregate accuracy while identifying weaker LOW recall and a printed-report label issue.

The technical evidence supports consistent dated forecast files, a historical demand database update and implemented web consumers. The provider-filter evidence supports explicit rules and polling, with limitations in time validation, date coverage, candidate assumptions and operational state handling. The component is consequently a connected research prototype whose broader planning value requires realistic temporal and user evaluation.

7 DISCUSSION

7.1 Interpretation of Findings

The comparison favours XGBoost on R squared, RMSE and MAE within the recorded synthetic experiment. Its advantage over Random Forest is small, and its percentage error is worse. The result should be interpreted as a local estimator choice, not proof of universally better forecasting. The category-mean baseline further places the model advantage in context.

The prototype's main practical strength is its consistent output representation. Daily estimates, weekly totals and bands are structured for web consumption. The request workflow similarly makes schedule outcomes and distances explicit. These interfaces can support future evaluation even when the current model and rule assumptions remain limited.

The findings also demonstrate the importance of checking labels and provenance. Misnamed classification rows can change which band appears weakest. Mixing forecast versions can change totals. Neither issue is resolved by presenting a more polished chart; the data and label mapping must be checked first.

7.2 Comparison With Literature Baselines and Existing Solutions

The implemented estimators are consistent with the tree-ensemble foundations reviewed in Chapter 2 [5], [6]. Their comparison remains specific to the generated task. The supplementary category-mean result illustrates why a task-specific baseline is necessary before attributing performance to model complexity.

Temporal forecasting practice highlights a limitation of the random split [7]. A later study should train on earlier observed bookings and evaluate a fixed future horizon, with feature values available at each origin. This design would answer a different and more operationally relevant question than predicting another randomly selected generated row.

Deterministic filtering is appropriate when eligibility rules are explicit and validated. The current method is transparent, but its permissive malformed-time handling and

coordinate defaults weaken that transparency in practice. A validated rule system should remain a baseline for any later learned recommender.

7.3 Research Contribution

The supported research contribution is an analysed experiment and an integrated prototype spanning demand estimation and operational candidate processing. The thesis documents how the synthetic target is constructed, how encoded features are used, how model selection is made and how numerical estimates become bands and weekly outputs.

Supplementary analysis contributes checks that were not printed as part of the original model comparison. These include simple baselines, correctly labelled class metrics, forecast-file consistency and selected schedule/weather boundaries. Their role is to clarify the evidence and improve future experimental design. They do not add a new real-world training population or live deployment result.

The component does not introduce a new tree-learning algorithm, a validated weather-risk model or a trained personalised provider-ranking model. Its research value should be expressed through the implemented workflow and the demonstrated understanding of its limitations.

7.4 Product Contribution

The product contribution is a reusable forecasting function, a shared weekly demand payload, an administrator timetable, a provider category view and a provider-filter processing path. These outputs make the component inspectable through both code and stored data. Keeping the encoder and feature order with the estimator supports inference consistency.

The filter can explain eligibility through working-hours status and provide a geographical distance rather than an unexplained rank. Weather advice adds contextual information at the request level. However, the advice does not establish a provider-specific suitability difference, and fallback behaviour needs explicit representation before operational use.

The product can be strengthened without replacing every algorithm. Strict request validation, forecast provenance, idempotent processing and transparent fallback labels may be more immediately valuable than a more complex recommender. Those priorities follow the observed defects and interface risks.

7.5 Limitations and Threats to Validity

7.5.1 Construct and External Validity

The demand target represents a simulation rule rather than independently observed marketplace activity. Generated histories, ratings and supply figures cannot establish behaviour of actual customers or providers. Five sampled districts do not provide empirical geographic coverage, and fixed Colombo inference does not support district-specific forecasting claims.

Demand bands depend on thresholds chosen for this prototype. They should not be interpreted as universally meaningful capacity categories. Likewise, computed schedule availability describes declared hours, not a confirmed absence of other appointments. These construct definitions must remain visible in both research reporting and the interface.

7.5.2 Internal and Statistical Validity

Target encoding before inner cross-validation introduces a leakage concern. The holdout encoder does not use holdout targets, but the same holdout is used to compare and select the model family. A final untouched assessment period and fold-contained preprocessing would strengthen evaluation. No repeated split, confidence interval or uncertainty estimate supports significance claims.

The random split is not a temporal forecasting test. Generated lag-named features do not follow an observed time sequence, and future scenario inputs are fixed. MAPE is influenced by small counts, explaining why model ordering differs across numerical metrics. Class-specific results are sensitive to explicit label order.

7.5.3 Software and Operational Validity

Invalid or reversed request windows can yield availability. Missing customer locations use defaults, and missing provider coordinates can remain in the eligible list. Weather advice can use the first forecast date when the requested date is absent. These behaviours can produce technically complete results with weaker operational meaning.

The filter performs separate result and status writes, without an atomic job claim. Multiple workers or interruption between writes could lead to duplicate processing or ambiguous state. The five-second polling value is a configuration, not a measured response-time guarantee. No load or concurrency benchmark resolves these concerns.

7.5.4 Reproducibility and Attribution

README metrics and weather thresholds differ from authoritative code and recorded results. Forecast snapshots for the same week contain different totals. The tested Python environment is not pinned, and precise winning model parameters are not printed in the reviewed output. These gaps limit faithful reproduction from the archive alone.

The archive includes a broad web application. Presence of source files does not determine personal authorship. The candidate should verify individual and shared contributions against group records and export the exact final artefact versions used in the thesis.

7.6 Implications

The principal implication is that model, software and interface evaluation must be treated together. A numerical improvement on synthetic labels does not make malformed schedules reliable. A successful historical database write does not make every later dashboard value current. Each layer requires its own evidence.

For the next iteration, realistic observed booking data and temporal evaluation should strengthen the demand study. In parallel, the request workflow should reject invalid

inputs, expose weather-date coverage, preserve processing states and distinguish live from fallback data. These changes would make the prototype's outputs easier to interpret and its claims easier to verify.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The proposed value is clearer service planning. Providers could inspect a category-specific weekly activity estimate, administrators could compare category patterns, and customers could receive a smaller list of schedule-eligible candidates with distance and weather context. These are intended benefits requiring user evidence, rather than measured increases in bookings or income.

A useful adoption study should identify decisions users actually make with each output. Providers may plan working hours, but simulated counts cannot be presented as guaranteed work. Customers may compare distances, but eligibility must be distinguished from booking confirmation. Administrators may inspect history, but the history should retain source and processing provenance.

8.2 Market and Competitive Position

The relevant comparison set includes manual schedule coordination, static provider directories, simple category-level demand averages and more sophisticated forecasting or recommendation approaches. The current component's differentiation is the connected presentation of demand scenarios and rule-based request processing within the available platform.

No completed market survey or verified competitor performance study is present in the artefacts. The component should therefore avoid claims of market leadership or unique commercial effectiveness. Its position can be evaluated through a pilot comparing decision time, correction frequency and user understanding against a simpler workflow.

8.3 Business or Sustainability Model

A future operating model would need to account for hosting, database traffic, forecast execution, weather requests, support and maintenance. Polling the complete provider, customer and request dictionaries can increase operating work as the

platform grows. A queue or scoped query could reduce unnecessary processing, but its benefit should be measured in a realistic environment.

No confirmed pricing, revenue, subscription decision or financial forecast is documented. The appropriate immediate goal is a sustainable prototype with clear ownership, secure configuration, reproducible model releases and monitored failures. Financial projections should follow evidence of adoption and operating cost.

8.4 Scalability Intellectual Property and Compliance

The scaling challenges concern both data volume and workflow state. Repeated full-node reads and per-request weather calls should be evaluated under realistic request rates. Cached weather data, event-driven processing, job claiming and timeouts are candidate improvements. The existing small demonstrations do not establish performance at marketplace scale.

The team should confirm ownership of code, synthetic datasets, model artefacts and shared interface modules. Dependency licences and the terms of external data services should be reviewed before distribution. Personal location data and credentials should be restricted, retained only as required and excluded from public demonstration materials.

These responsibilities are especially relevant when replacing synthetic data with real booking histories. Research approval, participant information and data handling should be defined before collection. The thesis does not claim that an unavailable approval or external certification has already been obtained.

8.5 Contribution to the Sustainable Development Goals

A plausible alignment is SDG 8, which concerns decent work and economic growth [13]. Better presentation of service opportunities and operational constraints could support participation in a digital service marketplace. The component's current experiments do not measure employment, income, job quality or inclusion outcomes.

Potential negative effects should also be evaluated. Providers without complete schedules or reliable digital access may be omitted, and inaccurate demand estimates could encourage poor planning. A pilot should examine who benefits and who is

excluded, alongside whether users understand the limitations of forecasts and eligibility labels. SDG contribution should be reported through measured outcomes when those observations become available.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

The component demonstrates a connected prototype for synthetic service-demand estimation and rule-based provider filtering. The original comparison selects XGBoost with R squared 0.882890, RMSE 3.423339 and MAE 2.926993 on 600 random holdout rows. The derived demand-band accuracy is 79.33 percent. A strong category-mean baseline and the generated target structure limit interpretation as a validated future-booking forecast.

The dated forecast files provide consistent seven-day category summaries, and the synchronisation notebook records one historical fourteen-category database update. The provider-filter code implements declared-schedule checks, great-circle ordering and weather advice. Its recorded listener run is idle, and supplementary checks identify important validation limitations. The research questions are answered at prototype level, with realistic temporal and user validation remaining open.

9.2 Achievement of Objectives

Table 9.1 maps the objectives to the available evidence. The mapping separates implementation achievement from stronger validation that remains necessary.

Table 9.1 Achievement of individual objectives

Objective	Evidence	Achievement and boundary
Create an explicit dataset	Seeded generator and 3000-row CSV.	Achieved for simulation; not real demand collection.
Compare regressors and bands	Model metrics, predictions and class checks.	Achieved for recorded experiment; validation limits remain.
Generate a seven-day forecast	Ninety-eight daily rows and fourteen summaries.	Achieved for dated fixed scenarios.
Implement provider filtering	Schedule, distance, weather and polling code.	Implemented; robustness and processed-request evidence incomplete.

Objective	Evidence	Achievement and boundary
Connect outputs and identify limits	Demand sync, web consumers and offline checks.	Demonstrated interfaces; full user validation pending.

The most defensible achievement is a documented processing and evaluation workflow. Its limitations identify a concrete path for further research rather than invalidating the usefulness of the artefact. Future claims should be aligned with the stronger tests that are actually completed.

9.3 Individual Research and Product Contributions

The research contribution is the design and analysis of a synthetic regression comparison, threshold-derived demand evaluation and scenario forecasting workflow. The product contribution is the saved inference package, shared demand contract and deterministic provider-processing path with related presentation modules.

The supplementary analyses clarify the role of category baselines, correct class-label interpretation and expose request-validation behaviour. The candidate must confirm individual task ownership and any shared assistance. That confirmation supports a thesis account focused on the actual work rather than the entire group application's source inventory.

9.4 Future Directions

The first priority is observed longitudinal booking data with a documented forecast origin and future evaluation window. Historical-order features should be constructed from earlier observations, and target encoding should be fitted within temporal folds. A final untouched period, category baselines, uncertainty estimates and horizon-specific errors would strengthen the demand study.

The second priority is robust request validation. Missing coordinates, reversed or invalid intervals, overnight schedules and out-of-range weather dates should produce explicit unresolved states. Candidate-category and booking-conflict checks should be

defined through shared contracts. Weather advice should retain its forecast date and heuristic status.

The third priority is reproducible and reliable operation: a model manifest, consistent documentation, versioned forecast history, atomic job claiming, idempotent writes, monitored failures and clear live/cache labels. A controlled user pilot can then measure decision quality and correction needs. These steps provide a realistic route from the current connected prototype to a stronger research and deployment claim.

REFERENCES

[1] Individual demand component, "Dataset generation, model comparison and exported evaluation," demand_model.ipynb, dataset/service_demand_prediction_dataset.csv and results/*.csv, supplied in New RP.zip, unpublished project artefacts, reviewed Oct. 10, 2026.

[2] Individual filtering component, "Provider schedule distance weather and request processing," filtering.ipynb, supplied in New RP.zip, unpublished project artefact, reviewed Oct. 10, 2026.

[3] Individual forecasting component, "Next-week scenario forecasting and database synchronisation," sync_firebase_demand.py, predict_next_week_demand.py, firebase_demand_sync.ipynb and dated forecast CSV files, supplied in New RP.zip, unpublished project artefacts, reviewed Oct. 10, 2026.

[4] Shared project web application, "Demand timetable, provider demand view, filtering history and database fetch interfaces," relevant React and TypeScript files in updated_WEB/WEB/src, supplied in New RP.zip, unpublished project artefacts, reviewed Oct. 10, 2026.

[5] L. Breiman, "Random forests," Machine Learning, vol. 45, pp. 5-32, 2001, doi: 10.1023/A:1010933404324. Available: https://link.springer.com/article/10.1023/A:1010933404324

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785-794, doi: 10.1145/2939672.2939785. Available: https://arxiv.org/abs/1603.02754

[7] R. J. Hyndman and G. Athanasopoulos, Forecasting Principles and Practice, 3rd ed., OTexts, 2021, sec. 5.10, "Time series cross-validation." Available: https://otexts.com/fpp3/tscv.html

[8] Category Encoders, "Target Encoder," official documentation. Available: https://contrib.scikit-learn.org/category_encoders/targetencoder.html. Accessed: Oct. 10, 2026.

[9] scikit-learn, "Common pitfalls and recommended practices," official documentation. Available: https://scikit-learn.org/stable/common_pitfalls.html. Accessed: Oct. 10, 2026.

[10] scikit-learn, "classification_report," official documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html. Accessed: Oct. 10, 2026.

[11] Open-Meteo, "Weather Forecast API," official documentation. Available: https://open-meteo.com/en/docs. Accessed: Oct. 10, 2026.

[12] Google Firebase, "Saving data," Realtime Database Admin documentation. Available: https://firebase.google.com/docs/database/admin/save-data. Accessed: Oct. 10, 2026.

[13] United Nations Department of Economic and Social Affairs, "Goal 8," Sustainable Development Goals. Available: https://sdgs.un.org/goals/goal8. Accessed: Oct. 10, 2026.

[14] Sri Lanka Institute of Information Technology, "Guidelines on documentation and submission of dissertations," supplied as Formating Standards.pdf; and "Individual thesis content," supplied as Individual Thesis Content.docx, undated institutional guidance.

GLOSSARY

Demand scenario. A model output produced using assumed future covariates rather than a fully observed marketplace state.

Forecast origin. The time at which a future prediction is issued and the available information is fixed.

Target encoding. A supervised numerical representation of categories using target-related statistics.

Declared-schedule eligibility. Compatibility with stored operating hours; it is distinct from confirmed booking availability.

Great-circle distance. Geographic distance along a spherical surface, used here as a proximity approximation rather than a road-route measure.

Provenance. Information identifying the source, version, period and procedure that produced an output.

Idempotent processing. A design in which retrying the same job does not create inconsistent duplicate effects.

APPENDIX A EXTENDED DATA AND EVALUATION EVIDENCE

A.1 Category and District Counts

Table A.1 contains the supplementary realised category counts. The frequencies describe the generated training table and should not be interpreted as market shares. Table A.2 contains the five district counts.

Table A.1 Generated service-category counts

Category	Records	Percentage
A/C Services	253	8.43
CCTV Installation	245	8.17
Carpenters	220	7.33
Ceiling Works	206	6.87
Cleaners	215	7.17
Cushion Works	195	6.50
Electricians	194	6.47
Masons	218	7.27
Painters	209	6.97
Plumbers	207	6.90
Technicians	191	6.37
Tile Workers	214	7.13
Vehicle Repairs	227	7.57
Well Services	206	6.87

Table A.2 Generated district counts

District	Records	Percentage

District	Records	Percentage
Colombo	582	19.40
Galle	563	18.77
Gampaha	643	21.43
Kalutara	584	19.47
Kandy	628	20.93

A.2 Candidate Hyperparameter Spaces

Table A.3 preserves the candidate lists used by the original RandomizedSearchCV experiment [1]. Each family samples fifteen configurations with five-fold evaluation. The candidate space contains 864 combinations for Random Forest and 6,480 for XGBoost. The experiment does not evaluate every combination. Exact selected parameter dictionaries should be exported from the original training environment for final reproducibility.

Table A.3 Original candidate hyperparameter lists

Model	Parameter	Candidate values
Random Forest	n_estimators	50, 100, 150, 200
Random Forest	max_depth	None, 10, 15, 20
Random Forest	min_samples_leaf	1, 2, 4
Random Forest	min_samples_split	2, 5, 10
Random Forest	max_features	sqrt, log2, 1.0
Random Forest	bootstrap	True, False
XGBoost	learning_rate	0.01, 0.03, 0.05, 0.1, 0.2
XGBoost	max_depth	3, 5, 7, 9
XGBoost	subsample	0.6, 0.8, 1.0
XGBoost	colsample_bytree	0.6, 0.8, 1.0

Model	Parameter	Candidate values
XGBoost	gamma	0, 0.1, 0.2
XGBoost	min_child_weight	1, 3, 5
XGBoost	n_estimators	100, 150, 200, 300

A.3 Supplementary Audit Procedure

The audit reads the dataset and exported predictions as CSV files. It splits dataset row indices with test_size 0.2 and random_state 42, then verifies that holdout actual targets match the prediction file. The global mean uses only training targets. The category mean groups training targets by service_category and maps the resulting means to holdout categories. The fixed-base reference uses the category constants in the generator.

The classification audit reads the stored actual and predicted band strings and explicitly sets labels to LOW, MEDIUM, HIGH for the confusion matrix and report. It does not regenerate bands from rounded counts because rounding near a threshold could differ from the original unrounded evaluation. Forecast checks sum each seven-day row and separately group the daily file by category.

The helper audit extracts only the pure functions needed for parsing times, checking declared schedules, calculating Haversine distance and interpreting a fabricated weather dictionary. Firebase initialisation, network acquisition and the listener invocation are not executed. The model pickle files are not loaded and the training pipeline is not rerun. These boundaries make the supplementary results reproducible without using live account access.

A.4 Proposed Additional Validation

Table A.4 lists important future checks. They are proposed cases, not additional completed experiments. Each should record inputs, expected behaviour, observed output, version and correction status when executed.

Table A.4 Proposed operational validation matrix

Area	Case	Desired behaviour
Time validation	Missing, invalid or reversed request interval	Reject or preserve explicit unresolved state.
Schedules	Overnight hours and existing bookings	Evaluate a defined date-aware availability model.
Coordinates	Missing, malformed or out-of-range values	Avoid silent location substitution.
Candidate contract	No category-qualified candidate identifiers	Preserve category requirements before ranking.
Weather coverage	Requested date absent from forecast	Return unavailable context for that date.
External service	Timeout, invalid JSON or incomplete arrays	Preserve error state and retry policy.
Concurrency	Two workers process the same request	Claim atomically and write idempotently.
Forecast freshness	Expired, cached or seeded payload	Label provenance and target period visibly.
Temporal model	Future observed booking period	Evaluate against baselines with valid forecast origins.
User interpretation	Count, band and availability explanations	Measure understanding and correction needs.

APPENDIX B ACADEMIC INTEGRITY AND AI USE DISCLOSURE

B.1 Assistance Used in Preparing This Thesis

ChatGPT with Codex assistance was used to inspect the supplied component archive and institutional guidance, draft and organise English thesis content, check official technical references, prepare figures and tables, compute the supplementary CSV baselines and class metrics, inspect isolated pure helper behaviour, and format and render the Word document. The original notebooks and exported files provide the historical model and integration results.

The representative request was to create an individual thesis for a group member using New RP.zip and the previously supplied individual thesis guide and formatting standards. Assistance did not collect real booking histories, recruit stakeholders, train a replacement model or perform a new live Firebase or weather request. Supplementary checks are identified in the methodology, implementation and results chapters.

B.2 Candidate Review and Attribution

Before signing the declaration, the candidate must review the technical account against their own work, confirm the exact artefact versions, verify references and supplementary calculations, and agree individual/shared contributions with the team. Any other assistance used during development or writing should be disclosed under the module's actual policy.

The cover requires the candidate's name, student ID, group ID, official project title, degree, specialisation, department, supervisors and submission month. Acknowledgements and signatures require completion by the relevant people. The presence of a local account name in notebook paths is not used to infer the candidate's identity.

B.3 Institutional Structure and Formatting

The document follows the supplied nine-chapter contents structure, including commercialisation and sustainability, and the institutional A4, margin, font and pagination guidance [14]. The supplied guide's later body heading list omits the commercialisation chapter, so the complete contents structure is retained here. The candidate should confirm any current individual-summary length requirement with the module supervisor.

APPENDIX C TECHNICAL INTERFACES AND WORKED EXAMPLES

C.1 Model Feature Order

Table C.1 preserves the feature order constructed in the notebook. The inference function selects saved feature_columns after encoding. The candidate should confirm that the delivered model artefacts correspond to this notebook version before release.

Table C.1 Ordered model feature schema

Order	Feature	Representation
1	day_of_week	Target-encoded category
2	month	Numerical
3	week_of_year	Numerical
4	service_category	Target-encoded category
5	district	Target-encoded category
6	city	Target-encoded category
7	rainfall_mm	Numerical
8	temperature_c	Numerical
9	humidity	Numerical
10	weekend	Numerical
11	holiday	Numerical
12	available_providers	Numerical
13	previous_day_orders	Numerical
14	last_7_day_avg	Numerical
15	last_30_day_avg	Numerical

Order	Feature	Representation
16	search_requests	Numerical
17	completed_orders	Numerical
18	cancelled_orders	Numerical
19	avg_provider_rating	Numerical
20	avg_distance_km	Numerical
21	year	Numerical
22	day	Numerical
23	week	Numerical
24	weekday	Numerical
25	day_of_year	Numerical

C.2 Worked Forecast Aggregation

For the exported Electricians scenario, Monday through Friday are 34 each and Saturday and Sunday are 35 each. The weekly total is 5 times 34 plus 2 times 35, giving 240. The average is 240 divided by 7, rounded to 34.3. Since the average exceeds 25, the weekly band is HIGH. This example explains aggregation of a model scenario, not measured customer orders.

For A/C Services, five weekday values of 23 and two weekend values of 26 produce total 167 and average 23.9. The weekly band is MEDIUM even though the individual weekend counts exceed 25. Weekly classification applies to the rounded average, while daily interpretation applies to each day's count. Consumers should preserve this distinction.

C.3 Request Interface and Eligibility Example

A generic request contains user_id, service_date, service_time with start_time and end_time, location_type and isNewRequest. Candidate identifiers may be supplied directly or through a result node. The current filter writes provider_ids,

evaluated_providers, weather_risk, recommendation, weather_summary and evaluated_at to output_results, then clears isNewRequest [2].

For a Monday provider schedule of 08:00 to 18:00, a request from 10:00 to 12:00 passes the current interval containment check. A request beginning at 07:00 fails. A reversed request from 17:00 to 10:00 also passes the current code, because the comparison lacks a separate end-after-start condition. This illustrates the defect documented in Table 5.1 and the need for validation before filtering.

C.4 Artefact Reconciliation

The thesis uses the notebook and exported metrics for the regression result and the internally consistent dated CSV pair for the August 3 to 9 scenario. The README's alternative model values and thresholds are treated as documentation discrepancies. The August 16 synchronisation notebook concerns a different target week and is not merged into that CSV scenario.

The original classification report's display-name issue is corrected through an explicit-label supplementary table. These reporting choices preserve provenance rather than editing the historical artefacts. A final release should regenerate notebook outputs, documentation, plots and CSVs from the same versioned run and preserve a release manifest.